

WEB SISTEMAK – LABURPENA 2026/02/06

Gaurko klasean **HTTP datu-bidalketa** landu dugu **Pythonen**, atzoko saioan Burp erabiliz editatu genituen eskaera gordinak abiapuntu bezala hartuta. Horrela, datu-bidalketa motak errepasatu ditugu: POST, datuak inprimaki formatuan bidaliz; POST, datuak JSON formatuan bidaliz; eta GET. Helburua da datu-bidalketa egiteko HTTP eskaeraren egitura ulertzea: datuak non txertatzen diren (momentuz, gorputzean POST erabiltzen bada eta URIan GET erabiltzen bada), zein goiburu gehigarri behar diren kasu bakoitzean (POSTek Content-Type eta Content-Length behar ditu), eta datuak formateatu eta kodetzea ezinbestekoa dela azpimarratzea (inprimaki formatuan `urllib.parse.urlencode(educia)` eta json formatuan `json.dumps(educia)`).

Praktikatzeko [3. asteko 2026/02/05 eskolako apunteetako](#) 12. gardenkiko ariketa egitea gomendatzen da: EHUko direktorioan “armentia” abizena duten irakasleen bilaketa egitea:

- <https://www.ehu.eus/bilatu/buscar/bilatu.php> HTML web orrialdearen iturri kodea aztertzea (“Ver código fuente de la página” aukera) inprimikiak (*<form* elementuak) zein metodo darabilen, datuak zein URIra bidaltzen diren eta datu-eremuen (*<input* elementuak) izenak zeintzuk diren jakiteko.
- HTTP eskaera Burp erabiliz bidaltzea.
- HTTP eskaera Pythonen programatzea.

Ondoren, **HTTP erantzunaren konpresioa** aztertu dugu adibide praktiko baten bidez. Pythonen programa bat egin dugu, zeinek EHUko webgunearen orrialde nagusia eskatzen duen. Erabiltzaileak CLI-tik erabaki dezake erantzuna konprimatuta nahi duen ala ez. Programaren erabilera honakoa da: `python konpresioa.py [gzip]`.

- Erabiltzaileak ez badu argumenturik adierazten, programak *Accept-Encoding: identity* goiburua bidaltzen du, edukia konprimatu gabe nahi duela adieraziz.
- Aldiz, *gzip* argumentua adierazten bada, *Accept-Encoding: gzip* goiburua bidaltzen da, zerbitzariari erantzuna konprimatuta bidaltzeko eskatuz.

Beraz, argi ikusten da goiburuak nola baldintzatu dezaketen zerbitzariaren erantzuna.

Jarraian, **karakteen kodifikazioari** buruz hitz egiten hasi gara. Ordenagailuek karaktereak kudeatzeko, lehenik eta behin karaktere edo sinbolo bakoitzari *Unicode* kode-puntu bat esleitzen diote. Beraz, kode-puntua karaktere baten adierazpen abstraktua da. Ondoren, *ASCII*, *Latin-1* edo *UTF-8* bezalako kodifikazio-taulek zehazten dute Unicode kode-puntu hori nola bihurtzen den byte-sekuentzia batean, hau da, nola gordetzen den memorian edo nola bidaltzen den sarean. Adibidez, honako Unicode kode-puntuak ditugu:

- $e \rightarrow U+0065$
- $\acute{e} \rightarrow U+00E9$

Kodifikazio-taula desberdinek honela bihurtzen dituzte Unicode balio horiek byteetara:

- **ASCII:**
 - $U+0065 \rightarrow 0x65$
 - $U+00E9 \rightarrow$ ez dago definituta
- **Latin-1 (ISO-8859-1):**
 - $U+0065 \rightarrow 0x65$
 - $U+00E9 \rightarrow 0xE9$
- **UTF-8:**
 - $U+0065 \rightarrow 0x65$
 - $U+00E9 \rightarrow 0xC3\ 0xA9$

Hemetik abiatuta, adibide praktiko landu dugu: ikasle batek “é” karakterea duen dokumentu bat gorde nahi du. Testu editorea ASCII kodifikazio-aula erabiltzeko konfiguratuta badago, hori ezinezkoa izango da, ASCIIk ez duelako karaktere hori onartzen. Aldiz, testu editorea UTF-8 kodifikazio-aula erabiltzeko konfiguratuta badago, “é” karakterea 0xC3 0xA9 byte-sekuentziarekin gordetzen da. Dokumentu hori beste ikasle bati bidaltzen bazaio, eta ikasle horren testu editorea Latin-1 kodifikazio-aula erabiltzeko konfiguratuta badago, 0xC3 bytea “Ã” bezala adieraziko da eta 0xA9 bytea “©” bezala. Adibide honek oso ondo erakusten du arazoa ez dagoela egindako kodifikazioan, baizik eta bidaltzaileak eta hartzaileak ez dutela kodifikazio-aula bera erabiltzen.